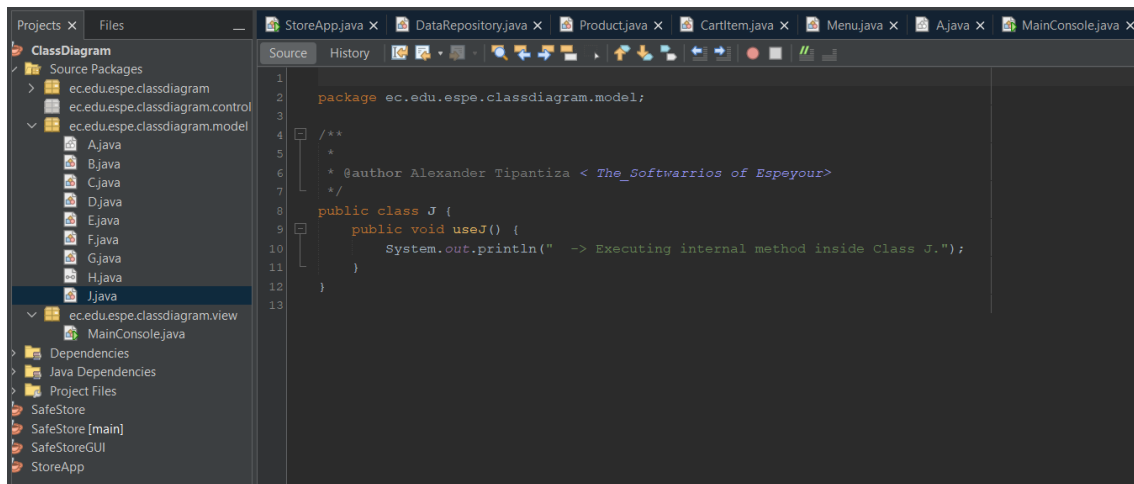
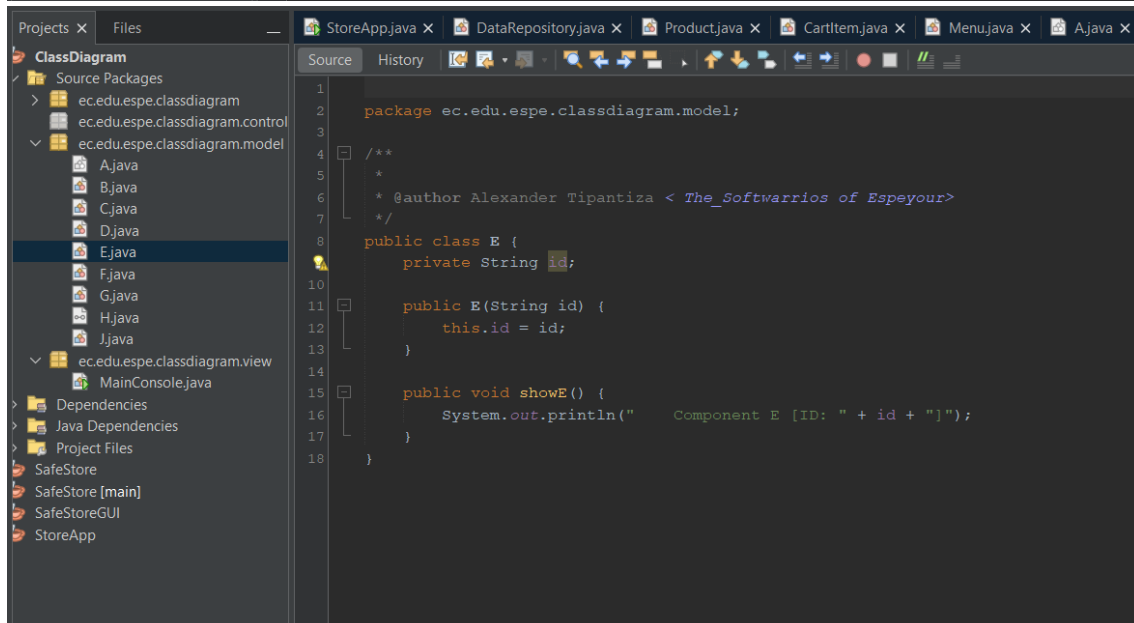


Evidence from the Class Diagram Theoretical Exam



This screenshot shows an IDE with the 'ClassDiagram' project selected in the left sidebar. The 'Source' tab is active, displaying the code for 'Class J'. The code is as follows:

```
1 package ec.edu.espe.classdiagram.model;
2
3
4 /**
5  *
6  * @author Alexander Tipantiza <The_Softwarrios of Espeyour>
7  */
8 public class J {
9     public void useJ() {
10         System.out.println(" -> Executing internal method inside Class J.");
11     }
12 }
13
```



This screenshot shows the same IDE with the 'ClassDiagram' project selected. The 'Source' tab is active, displaying the code for 'Class E'. The code is as follows:

```
1 package ec.edu.espe.classdiagram.model;
2
3
4 /**
5  *
6  * @author Alexander Tipantiza <The_Softwarrios of Espeyour>
7  */
8 public class E {
9     private String id;
10
11     public E(String id) {
12         this.id = id;
13     }
14
15     public void showE() {
16         System.out.println("    Component E [ID: " + id + "];");
17     }
18 }

```

Projects X Files — StoreApp.java X DataRepository.java X Product.java X CartItem.java X Menu.java X A.java X MainConsole.java X

ClassDiagram

- Source Packages
 - ec.edu.espe.classdiagram
 - ec.edu.espe.classdiagram.control
 - ec.edu.espe.classdiagram.model
 - A.java
 - B.java
 - C.java
 - D.java
 - E.java
 - F.java
 - G.java
 - H.java
 - J.java
 - ec.edu.espe.classdiagram.view
 - MainConsole.java
- Dependencies
 - Java Dependencies
 - Project Files
 - SafeStore
 - SafeStore [main]
 - SafeStoreGUI
 - StoreApp

```
1 package ec.edu.espe.classdiagram.view;
2 import ec.edu.espe.classdiagram.model.*;
3 /**
4  *
5  * @author Alexander Tipantiza <The_Softwarewarrios of Espeyour>
6  */
7
8 public class MainConsole {
9     public static void main(String[] args) {
10         System.out.println("=====");
11         System.out.println("        CLASS DIAGRAM EXECUTION OUTPUT (ALEXANDER.)        ");
12         System.out.println("=====\\n");
13
14         System.out.println("--- 1. Testing Interface (H), Realization (G) and Dependency (J) ---");
15         G objectG = new G();
16         J objectJ = new J();
17
18         objectG.operationH();
19         objectG.dependOnJ(objectJ);
20         System.out.println();
21
22         System.out.println("--- 2. Generalization / Specialization (Inheritance) Hierarchy ---");
23
24         B childB = new B("Instance_B_");
25         C childC = new C("Instance_C_");
26         D childD = new D("Instance_D_");
27
28         childB.displayBehavior();
29         childC.displayBehavior();
30         childD.displayBehavior();
31         System.out.println();
32
33         System.out.println("--- 3. Association Testing ---");
34         childB.addAssociationH(objectG); // B stores a reference to Interface H implemented via G
35         System.out.println("=> Structural Association between Class B and Interface H successfully established.");
36     }
37 }
```

Projects X Files — StoreApp.java X DataRepository.java X Product.java X CartItem.java X Menu.java X A.java X

ClassDiagram

- Source Packages
 - ec.edu.espe.classdiagram
 - ec.edu.espe.classdiagram.control
 - ec.edu.espe.classdiagram.model
 - A.java
 - B.java
 - C.java
 - D.java
 - E.java
 - F.java
 - G.java
 - H.java
 - J.java
 - ec.edu.espe.classdiagram.view
 - MainConsole.java
- Dependencies
 - Java Dependencies
 - Project Files
 - SafeStore
 - SafeStore [main]
 - SafeStoreGUI
 - StoreApp

```
1 package ec.edu.espe.classdiagram.model;
2 import java.util.ArrayList;
3 import java.util.List;
4
5 /**
6  *
7  * @author Alexander Tipantiza <The_Softwarewarrios of Espeyour>
8  */
9
10 public abstract class A {
11     protected String name;
12     protected List<A> subPartsA;
13
14     public A(String name) {
15         this.name = name;
16         this.subPartsA = new ArrayList<>();
17     }
18
19     public abstract void displayBehavior();
20
21     public void addSubPartA(A child) {
22         if (child != null) {
23             this.subPartsA.add(child);
24         }
25     }
26
27     public String getName() {
28         return name;
29     }
30 }
```

```
Output X
Run (StoreApp) X Run (MainConsole) X
=====
CLASS DIAGRAM EXECUTION OUTPUT (ALEXANDER.)
=====

--- 1. Testing Interface (H), Realization (G) and Dependency (J) ---
Class G: Formally implementing Interface H (Realization).
Class G: Temporarily depending on a J object to work (Dependency)...
-> Executing internal method inside Class J.

--- 2. Generalization / Specialization (Inheritance) Hierarchy ---
I am Object B ('Instance_B_'), inherited from A. Managing structural associations with H.
I am Object C ('Instance_C_'), inherited from A. Aggregating E objects (Max 3).
I am Object D ('Instance_D_'), inherited from A. Handling Composition with F and Aggregation with E.

--- 3. Association Testing ---
-> Structural Association between Class B and Interface H successfully established.

--- 4. Aggregation (Weak) vs Composition (Strong) ---
Aggregated contents inside Class C:
  Component E [ID: E-Alpha]
  Component E [ID: E-Beta]

Detailed internal architecture inside Class D:
-> Aggregated elements from E (External parts):
  Component E [ID: E-Beta]
  Component E [ID: E-Gamma]
-> Composed elements from F (Strictly owned life cycle):
  Internal Part F [Type: Core Memory Unit]
  Internal Part F [Type: Mathematical Processor F]

=====
UML Simulation Finished Successfully
=====

BUILD SUCCESS
=====

Run (StoreApp) X
```